

SmartJourney: Local Setup Guide

How to run the complete system on your own computer, build it, and run its tests. If you only want to *use* SmartJourney, the live version is at <https://aismartjourney.vercel.app>, and you don't need any of this.

The commands are for **Windows PowerShell**, with Git Bash for the `.sh` scripts. On macOS or Linux, use the same commands with `/` paths and `source .venv/bin/activate`.

1. What runs where

#	Component	Port	Folder
0	Docker: PostgreSQL + PostGIS, Redis, Keycloak (+ its own DB), Mailpit	5432, 6379, 8081, 8025	backend/
1	NestJS API	3001	backend/
2	AI backend (FastAPI)	8000	ai-backend/
3	Next.js web app	3000	frontend-web/

Start them **in this order**, because each one needs the ones before it. The web app talks only to the NestJS API, and the API calls the AI backend.

2. Prerequisites

Tool	Version
Docker Desktop	Recent (Compose v2). It must be running.
Node.js	22 or newer (CI uses 22 for the web app and 26 for the API)
Python	3.12 or newer
Git (with Git Bash on Windows)	Any recent version
A Google Gemini API key	Free from https://aistudio.google.com . Needed for the AI planner. Without it, the planner uses its rule-based fallback.

Free ports: 3000, 3001, 8000, 8081, 8025, 5432, 6379.

3. Get the code

From the submission zip, use the folder `1_Source_Code/`. Or clone the repositories into one folder:

```
mkdir SmartJourney; cd SmartJourney
git clone https://github.com/SmartJourney-SmartTourismProject/backend.git
git clone https://github.com/SmartJourney-SmartTourismProject/ai-backend.git
git clone -b new-main https://github.com/SmartJourney-SmartTourismProject/frontend-web.git
```

(frontend-web 's working branch is `new-main` .)

4. Environment files

Each service reads a `.env` file that is **not** in git. Create each one from its template:

```
copy backend\.env.example      backend\.env
copy ai-backend\.env.example    ai-backend\.env
copy frontend-web\.env.example  frontend-web\.env.local
```

Generate the secrets. Run this once **for each** secret, and use a different output each time:

```
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
```

Fill them in. **Values on the same line of this table must be identical:**

Value	backend/.env	ai-backend/.env	frontend-web/.env.local
Web client secret (generated)	KEYCLOAK_WEB_CLIENT_SECRET		KEYCLOAK_CLIENT_SECRET
Settings encryption key (generated)	SETTINGS_ENCRYPTION_KEY	SETTINGS_ENCRYPTION_KEY	
Internal API token (generated)	INTERNAL_API_TOKEN	INTERNAL_API_TOKEN	
Database URL	DATABASE_URL	DATABASE_URL (same value)	
next-auth secret (generated)			NEXTAUTH_SECRET
Gemini key		GEMINI_API_KEY	

Leave everything else at its default. In particular, `backend/.env` must keep `PORT=3001` , and `KEYCLOAK_ISSUER` stays `http://localhost:8081/realms/smartjourney` everywhere. Optional keys (`GROQ_API_KEY` , `OPENWEATHER_API_KEY` , `REDIS_URL` , and so on) add features, and the system works without them.

5. Start the infrastructure

```
cd backend
docker compose up -d
```

Wait until Keycloak is ready (about 30–60 seconds on the first start). This URL should return JSON:

```
curl http://localhost:8081/realms/smartjourney/.well-known/openid-configuration
```

The first start imports the `smartjourney` realm (clients, roles, password policy, login theme). It contains **no users**.

Then, from `backend/` in **Git Bash**, give the API's Keycloak service account its admin rights, which the Admin → Users tab needs:

```
sh keycloak/grant-service-account-roles.sh
```

It prints a client secret. Put it in `backend/.env` as `KEYCLOAK_ADMIN_CLIENT_SECRET`.

6. Database: schema and data

Option A (recommended): load the bundled data. Use this if you have the `demo-data` folder from the submission zip (`6_Executable/demo-data/` contains `sj.dump`, `fees.csv` and `fees.cols`). It holds all 25 districts, the verified listings, photos, events, entry fees and the knowledge base. It has no user accounts. In **Git Bash**, from `backend/` :

```
bash db/demo-data/restore.sh "/path/to/6_Executable/demo-data"
```

It prints row counts at the end. The dump already includes the migration history.

Option B: empty schema only. This works, but the planner has no places to recommend:

```
python -m venv ..\ai-backend\.venv
..\ai-backend\.venv\Scripts\pip install -r ..\ai-backend\requirements.txt
..\ai-backend\.venv\Scripts\python.exe db\migrate.py --status
..\ai-backend\.venv\Scripts\python.exe db\migrate.py
```

Either way, run `db\migrate.py --status` afterwards. It should report **nothing pending**.

7. Start the NestJS API (port 3001)

```
cd backend
npm install
npx prisma generate      # required on a fresh clone
npm start
```

It's ready when the log says `Nest application successfully started` . Health check:
`http://localhost:3001/health`

8. Start the AI backend (port 8000)

```
cd ai-backend
python -m venv .venv # skip if you already made it in step 6
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m uvicorn main:app --host 0.0.0.0 --port 8000
```

It's ready when the log says `Application startup complete` . The API docs are at
`http://localhost:8000/docs`.

If PowerShell blocks the activation script, run this once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` .

9. Start the web app (port 3000)

```
cd frontend-web
npm install
npm run dev
```

Open `http://localhost:3000`.

10. Accounts

Local Keycloak starts with **no users**. Create a traveler and an admin.

Admin console: `http://localhost:8081`, user `admin` , password `admin` . Switch to the realm **smartjourney**.

1. **Users** → **Add user**: enter a username and an email, set **Email verified** to **On**, then **Create**.
2. **Credentials** → **Set password**: 8–12 characters, with upper case, lower case, a number and a symbol. Set **Temporary** to **Off**.
3. For the admin account only: **Role mapping** → **Assign role**, choose **Filter by realm roles**, then `admin` .

Or register through the web app (**Sign up**). The verification email goes to the local **Mailpit** inbox at `http://localhost:8025`, not to a real inbox. Click the link within **5 minutes**.

11. Building

Component	Command	Output
NestJS API	<code>npm run build</code> (in <code>backend/</code>)	<code>dist/</code> . Run it with <code>npm run start:prod</code>

Component	Command	Output
Web app	<code>npm run build</code> , then <code>npm start</code> (in <code>frontend-web/</code>)	<code>.next/</code> production build
Docker images (production)	<code>docker build -t smartjourney-backend backend</code> and <code>docker build -t smartjourney-ai ai-backend</code>	Images like those the CI pushes to GHCR

The production deployment (Vercel + AWS EC2) is described in `4_System_Overview_and_Deployment.pdf` and, step by step, in `backend/deploy/MANUAL_SETUP.md`.

12. Running the tests

Suite	Where	Command	Needs
AI backend unit and integration	<code>ai-backend/</code>	<code>python -m pytest -m "not external" -q</code>	Nothing. External services are mocked (826 tests)
AI backend golden end-to-end scenarios	<code>ai-backend/</code>	<code>python scripts/e2e_check.py</code>	The full stack running
API unit tests	<code>backend/</code>	<code>npm test</code>	Nothing (146 tests)
API e2e, access control, security	<code>backend/</code>	<code>npm run test:e2e</code>	Docker infrastructure
Web unit tests	<code>frontend-web/</code>	<code>npm test</code>	Nothing (18 tests)
Web browser journeys	<code>frontend-web/</code>	<code>npx playwright install</code> once, then <code>npm run test:e2e</code>	The full stack running
Accessibility	<code>frontend-web/</code>	<code>npm run test:a11y</code>	The full stack running
Lint	<code>backend/</code> , <code>frontend-web/</code>	<code>npm run lint</code>	Nothing

The counts are from 2026-10-03. The test strategy and results are in the `Documentation/Testing/` folder.

13. Troubleshooting

Symptom	Fix
Module not found: Can't resolve 'next-auth/providers/keycloak'	Run <code>npm install</code> in <code>frontend-web</code> .
'keycloak_id' does not exist in type 'app_userWhereUniqueInput'	The generated Prisma client is stale. Run <code>npx prisma generate</code> in <code>backend</code> .
Sign-in works but the API returns 500 on a missing column	A migration is pending. Run <code>db\migrate.py</code> (step 6).
The Keycloak container won't start, or compose complains about blank variables	<code>backend/.env</code> is missing the <code>KEYCLOAK_DB_*</code> block. Compare it with <code>.env.example</code> .

Symptom	Fix
Sign-in loops or fails with a client error	<code>KEYCLOAK_CLIENT_SECRET</code> (web) $\neq$ <code>KEYCLOAK_WEB_CLIENT_SECRET</code> (API), or the web app isn't on port 3000.
Admin → Users says "Keycloak rejected the request (403)"	Run <code>keycloak/grant-service-account-roles.sh</code> and set <code>KEYCLOAK_ADMIN_CLIENT_SECRET</code> (step 5), then restart the API.
The chat says "Something went wrong reaching the trip planner"	Check the AI backend's log. Usually the LLM free-tier limits (Gemini 429/503, Groq 413/429). Retry later, or add a second provider key. The rule-based fallback still produces plans.
Plans are empty	No verified listings. Load the bundled data (step 6, option A) or approve listings under Admin → Listings.
AI backend returns 401 <i>bad internal token</i>	<code>INTERNAL_API_TOKEN</code> differs between <code>backend/.env</code> and <code>ai-backend/.env</code> .
No verification email arrives	That's expected locally. Open Mailpit at http://localhost:8025 .
The password is rejected	The realm policy allows at most 12 characters .
Realm changes disappear after <code>docker compose down -v</code>	<code>-v</code> deletes the volumes. The realm is re-imported, but users and data are lost.
"Google sign-in" doesn't work locally	It needs your own Google OAuth client (<code>GOOGLE_SIGNIN_CLIENT_ID/SECRET</code> in <code>backend/.env</code>). Use email and password instead.